

HOMEWORK 1 REPORT

Pac-man game implementation and heuristics

Umair Ahmad

Student ID: S031654

Department of Computer Science

umair.ahmad@ozu.edu.tr

CS 451: Introduction to Artificial Intelligence — Spring 2026

Abstract

Implementation of generic search algorithms which are called by Pacman agents to find a path from some starting point to the goal state. Algorithms are split into 2 main categories, Uninformed (blind) and informed (heuristic) search algorithms. Depth-first search (Dfs), Breath-first search (Bfs) and Uniform-cost search (UCS) are examples of uninformed search algorithms, whereas, A* star is for heuristic search.

Keywords: Dfs, Bfs, UCS, A* search

1 Introduction

In this project, pac-man agent has to find path from a given starting location and collect food effectively through the maze game. General search algorithms (Dfs, Bfs, UCS and A*) were implemented and applied to different game cases.

The implementation is mainly done in search.py class containing Generic search algorithms and search_agent.py class with search problem definitions and heuristic functions.

2 Background

The Algorithms applied to the game were based on graph search approach to prevent infinite loop issue, where in the chosen node is expanded and the resulting node is added to the frontier only if chosen node was not in visited set().

However, the A* search is implemented in a different way, it allows the same chosen node to be checked multiple time in search of the case where the future value cost is less than the stored future value.

The Heuristics in search_agent.py heavily rely on the correctness of these algorithm and should be admissible and consistent.

3 Methodology

- Graph Search:
Expand the chosen node and add the resulting nodes to the frontier only if not in the frontier or explored set, as forgetting the history causes repeating.

Algorithm 1: GRAPH-SEARCH(*problem*, *fringe*) returns a solution, or failure

```
closed  $\leftarrow \emptyset$ 
fringe  $\leftarrow$  INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
loop
  if fringe is empty then
    return failure
  end if
  node  $\leftarrow$  REMOVE-FRONT(fringe)
  if GOAL-TEST[problem](STATE[node]) then
    return SOLUTION(node)
  end if
  if STATE[node]  $\notin$  closed then
    add STATE[node] to closed
    fringe  $\leftarrow$  INSERTALL(EXPAND(node, problem), fringe)
  end if
end loop
```

- Depth First Search (DFS):
Uninformed search strategies like DFS use only the information available in the problem definition. In DFS, we expand deepest unexpanded node, keep nodes in a Stack and implement LIFO (least in first out).
- Breath First Search (BFS):
In BFS, we expand the shallowest node first, wherein the Fringe/ frontier is a FIFO queue e.g; new successor goes go the end of the queue. For reducing time complexity in general graph-search approach, the goal state test is carried out on the node upon being popped from the queue, rather than when it is selected for expansion.
- Uniform-cost search (UCS):
In UCS, we expand least-cost unexpanded node instead of the shallowest node, the fringe is Priority Queue ordered by path cost. The least cost node is always popped. UCS is equivalent to BFS if step-cost are all equal. The goal test is applied to a node when it is selected for expansion rather than it is first generated.

- A* search:

The idea behind informed search algorithms like A* search is to avoid expanding paths that are already expensive. It is the gold standard for path finding.

$$f(n) = g(n) + h(n) \quad (1)$$

$g(n)$ = cost so far to reach n .

$h(n)$ = estimated cost from n to goal.

$f(n)$ = estimated total cost of path through n to goal.

If the heuristic is "admissible" (never overestimates the distance), A* is guaranteed to find the most efficient path.

4 Implementation

The implementation was done in Python by modifying the files `search.py` and `search_agents.py`. In `search.py`, the generic graph-search algorithms Depth-First Search, Breadth-First Search, Uniform-Cost Search, and A* Search were implemented by storing search states together with the path of actions that leads to them. DFS uses a stack, BFS uses a queue, and UCS/A* use a priority queue.

In `search_agents.py`, problem-specific components were implemented for the Pac-Man environment. For the `CornersProblem`, the state representation was extended to include both Pac-Man's current position and the set of visited corners. For informed search, admissible heuristics were designed for both the corners problem and the food search problem. In addition, the closest-dot agent was implemented by defining a goal test for any food location and repeatedly applying search until all food was eaten.

An important implementation decision was to use graph search rather than tree search in order to avoid revisiting states unnecessarily. Another key design choice was to keep the search states compact but informative enough to encode progress, especially in the corners and food problems.

5 Results

The implemented algorithms were tested using the provided `pytest` test cases and Pac-Man layouts. DFS, BFS, UCS, and A* were all able to return valid action sequences for the corresponding search problems. BFS produced shortest paths in terms of number of actions, while UCS and A* considered path cost and heuristic information respectively.

For the `CornersProblem`, the state representation and successor generation correctly tracked visited corners and allowed Pac-Man to reach all four corners. The corners heuristic improved the efficiency of A* compared to uninformed search by reducing the number of expanded nodes. For the food search problem, the heuristic also guided the search more effectively than uniform-cost search, although the quality of the heuristic directly affected the expansion count.

Overall, the results show that informed search performs better than uninformed search when the heuristic is admissible and informative. In my implementation, the strongest improvements were observed in the corners and food-related

Listing 1: A heuristic for the `CornersProblem`

```

1  def corners_heuristic(state: Any,
    problem: CornersProblem) -> int:
2
3      _corners = problem.corners
4      _walls = problem.walls
5      position, visited_corners = state
6      visited_corners_list = list(
            visited_corners)
7      un_visited_corners = []
8
9      count = 0
10     for i, item in enumerate(
            visited_corners_list):
11         if item == False:
12             corner_i = _corners[i]
13             un_visited_corners.append((
                corner_i))
14             count += 1
15     if not count:
16         return 0
17
18     all_costs = []
19     permutations = list(itertools.
            permutations(un_visited_corners))
20
21     for tups in permutations:
22         costs = 0
23         for item in tups:
24             xy1 = position
25             xy2 = item
26             costs += (abs(xy2[0] - xy1
                [0]) + abs(xy2[1] - xy1
                [1]))
27             position = item
28             all_costs += [costs]
29
30     return min(all_costs)

```

tasks, where the search space is much larger than simple position search.

6 Discussion

The results confirm the expected behavior of the search algorithms. DFS is simple and can find a solution quickly, but it does not guarantee an optimal path. BFS guarantees the shortest path when all actions have equal cost, while UCS generalizes this idea to varying path costs. A* combines the actual path cost with heuristic guidance, which makes it more efficient when the heuristic is well designed.

In the corners problem, the challenge was representing both Pac-Man's position and the visited-corner information. In the food problem, the main challenge was designing a heuristic that remains admissible and consistent while still being informative.

A limitation of my approach is that heuristic quality still depends on simple distance-based estimates, which may ignore some maze structure. Future work could improve the heuristics further by caching distances more effectively.

Table 1: Tests summary

Question	Tests Passed	Performance / Grade
Q1: Depth-First Search	6/6	Passed all tests
Q2: Breadth-First Search	6/6	Passed all tests
Q3: Uniform-Cost Search	6/6	Passed all tests
Q4: A* Search	6/6	Passed all tests
Q5: Corners Problem	5/5	Passed all tests
Q6: Corners Heuristic	6/6	1007 nodes on <code>mediumCorners</code> → 9/9
Q7: Food Heuristic	5/5	4197 nodes on <code>trickySearch</code> → 12/12
Q8: Closest Dot Search	5/5	Passed all tests

Table 2: Selected Pac-Man search results

Method	Layout / Problem	Path Cost	Nodes Expanded	Score	Notes
DFS	<code>tinyMaze</code>	10	15	500	<code>SearchAgent</code>
DFS	<code>mediumMaze</code>	130	146	380	<code>SearchAgent</code>
DFS	<code>bigMaze</code>	210	390	300	<code>SearchAgent</code>
BFS	<code>mediumMaze</code>	68	269	442	<code>SearchAgent</code>
BFS	<code>bigMaze</code>	210	620	300	<code>SearchAgent</code>
UCS	<code>mediumMaze</code>	68	269	442	<code>SearchAgent</code>
A*	<code>bigMaze (Manhattan)</code>	210	549	300	<code>SearchAgent</code>
BFS	<code>tinyCorners</code>	28	252	512	<code>CornersProblem</code>
BFS	<code>mediumCorners</code>	106	1966	434	<code>CornersProblem</code>
A*	<code>mediumCorners</code>	106	901	434	<code>AStarCornersAgent</code>
A*	<code>trickySearch</code>	60	4137	570	<code>AStarFoodSearchAgent</code>
Closest Dot Search	<code>bigSearch</code>	350	—	2360	<code>ClosestDotSearchAgent</code>

7 Conclusion

In this Assignment, generic search algorithms and Pac-Man-specific search problems were implemented successfully. It demonstrated the importance of state representation, successor generation, and heuristic design in solving larger search problems efficiently.

The final implementation shows that while uninformed methods are useful as a baseline, heuristic-based search is significantly more practical for complex tasks such as visiting all corners or collecting all remaining food.